CS4005: Trusted Computing and Security Models

CIA - 2

Priyajit Biswal
Registration No: 23011102068
GitHub Repository: priyajitbiswal/trusted-computing-tpm-lab

Part A: Conceptual Questions (10 Marks)

1. TPM Architecture and Measured Boot

A Trusted Platform Module (TPM) is a secure cryptographic coprocessor that is usually attached to the motherboard. This chip contains a hardware Root of Trust that is responsible for generating and securing encryption keys, measuring the system's integrity during the boot process, and ensuring the computer's integrity.

The main parts of this security module are the following:

- Cryptographic coprocessor that performs encryption, decryption, and hashing operations (e.g., RSA, AES, SHA-256) to keep the encryption keys inside the TPM instead of the system memory, which can be accessed by malicious software.
- Non-volatile memory (NVRAM) stores cryptographic keys, the endorsement key (EK) that is used for manufacturing, owner passwords, and monotonic counters that prevent downgrading.
- The platform configuration registers (PCRs) are special memory registers that hold hashes of measured software, which is used to check the integrity of the system during the boot process. These registers are updated with cryptographic extend operations and should not be modified directly.
- Random number generator (RNG) ensures that high-quality random numbers are generated to protect cryptographic keys, session keys, and other security-related information.

- The authorization logic is responsible for granting access to the restricted keys and data stored within the TPM.

Measured Boot and PCRs: Measured Boot is the process of measuring (hashing) each component of software during boot before executing it. The fundamental concept is to 'measure before you execute'. Beginning at a read-only hardware block called the Core Root of Trust for Measurement (CRTM), the firmware measures the UEFI, which measures the bootloader, which in turn measures the operating system kernel.

Each measurement is recorded into PCR using the extend operation:

PCR Extend Operation: PCR_new = SHA256(PCR_old + New_Measurement)

Since this formula combines the old PCR value with a hash of the new component, the order in which they are combined is essential. In the case of malware attempting to compromise the operating system, any changes to the bootloader, kernel, or configuration files, will lead to a different PCR hash than expected.

Measured Boot vs. Secure Boot: Secure Boot prevents computer from starting if unauthorized or unsigned files are found. Measured Boot does not stop the boot process, but rather records a tamper-evident cryptographic log of everything that loaded into the PCRs. This log may later be used by remote attestation or to seal (protect) encryption keys.

2. TPM-Based Remote Attestation

The remote attestation process consists of several steps, which allow the client computer to assure the TPM server about the validity of its hardware and the correctness of the boot process. It should be noted that remote attestation is generally based on a simple principle and can be achieved in four basic steps, namely:

1. Generating an asymmetric Attestation Identity Key (AK) by the Trusted Platform Module (TPM), whose validity is guaranteed by the TPM’s endorsement key.

2. Sending a remote attestation request, which contains a random number to prevent replay attacks.

3. Computing a TPM quote, which includes the TPM's hash values stored in specific PCR registers and the random number received from the server in step 2, and signing this information with the AK generated in step 1.

4. Verifying the quote's signature by comparing the report's PCR values with a list of expected values and checking the nonce value to ensure that the client did not use a previous response.

Practical Scenario: When a user connects to a corporate server (e.g., a bank or hospital's intranet) via a notebook computer, the server first queries the Trusted Platform Module (TPM) to obtain a TPM Quote. If this laptop boots up normally using the designated operating system and bootloader, then the TPM PCR values would be in a permitted range that matches the server's expectations. On the other hand, if a laptop was compromised or infected with a rootkit, its measured kernel hash would be different, thus, the server would deny it access to critical data.

3. Trusted Execution Environment (TEE)

A Trusted Execution Environment (TEE) is isolated, secure area inside the main processor (CPU) that isolates the trusted applications from the rest of the operating system (such as Android or Windows). This isolation makes it hard to detect malicious programs and processes running in the TEE even by someone who has full root or administrator access to the main operating system.

Using ARM TrustZone as an example, the main components are the following:

- Normal World and Secure World – processor hardware is logically split into two execution environments. The Normal world is for regular applications and the operating system while the Secure World is for security-sensitive parts of the software.

- Non-Secure (NS) bit – a signal on the processor bus that tells the hardware whether a memory address or peripheral belongs to the Normal or Secure World. Hardware memory management then prevents the Normal World from reading the memory of the Secure World.

- Secure Monitor (EL3) – a small piece of software that runs at the highest privilege level in the CPU. It is responsible for switching between different execution modes, including the Secure and Normal World.

- Trusted OS and Applications – special versions of mobile operating systems (such as OP-TEE) and software that run inside the TEE. These are used for specialized tasks such as biometric authentication or storing and processing payment credentials.

Operational Flow:

1. User Request: A banking application running on the Normal World requests fingerprint verification to authorize a transaction.

2. Context Switch: The operating system makes a Secure Monitor Call (SMC) by executing an SMC instruction. The CPU saves the context of the Normal World and switches to the Secure Monitor.

3. Isolation: The Secure Monitor saves any registers used by the Normal World and zeroes any temporary CPU registers to prevent any information from being leaked before switching to the Secure World.

4. Execution: The biometric Trusted Application within the Secure World then compares the presented fingerprint with the stored fingerprint template within the Secure World's memory.

5. Return: The Secure World only returns a simple "Match / No Match" response to the Normal World while keeping the fingerprint data confidential.

4. TPM vs. TEE

Both TPM and TEE are used to protect computers and devices, but they operate at different points in the system architecture:

Aspect	**Trusted Platform Module (TPM)**	**Trusted Execution Environment (TEE)**
Location	Separate chip on the motherboard or dedicated firmware module.	Built directly inside the main CPU core (like ARM TrustZone).
What it Runs	Fixed cryptographic functions and key storage; cannot run general software.	Runs full secure programs (Trusted Applications) at processor speed.

Memory Size	Very small internal storage (a few kilobytes for keys and PCRs).	Can use megabytes of RAM, protected by hardware memory isolation.
Main Purpose	Checking boot integrity, platform measurements, and unlocking disk keys.	Protecting sensitive code and data while the operating system is actively running.
Common Uses	BitLocker disk encryption, Measured Boot, remote attestation.	Fingerprint/Face unlock, mobile wallet payments, 4K video DRM.

How They Work Together:

TPM and TEE are complement of each other. The TPM makes sure that the machine was booted in an untampered state and unlocks the root encryption keys. The TEE is there to protect the machine's real-time sensitive tasks (such as biometrics) from being compromised by malware in the operating system.

Part B: Practical and Analytical Tasks (10 Marks)

TPM-Based Secure Key Storage and Data Sealing

This lab was completed using the software TPM (swtpm) and the tpm2-tools package on Ubuntu Linux. The task was to generate an encryption key, seal some sensitive information to a PCR register, and demonstrate that the TPM can unseal the information when the PCR is measured but not when it has been tampered.

1. Environment Setup and TPM Verification

Commands Executed: To set up the TPM environment safely, swtpm was launched in socket mode on ports 2321 and 2322. The following commands were executed:

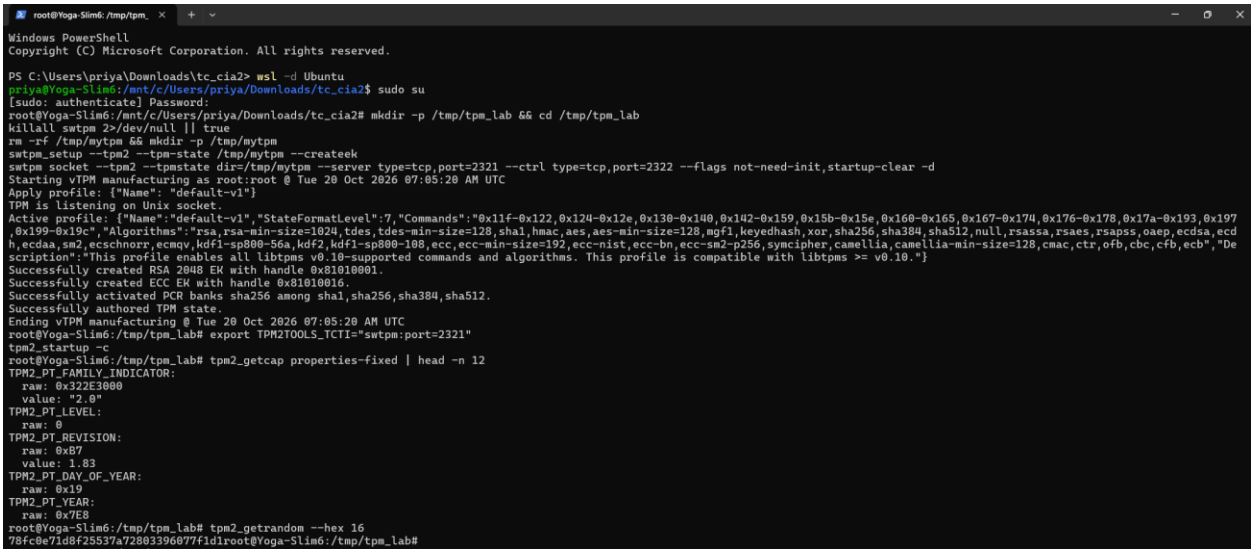
```
# Setup directory and start swtpm in daemon mode
mkdir -p /tmp/tpm_lab && cd /tmp/tpm_lab
killall swtpm 2>/dev/null || true
rm -rf /tmp/mytpm && mkdir -p /tmp/mytpm
swtpm_setup --tpm2 --tpm-state /tmp/mytpm --createek
swtpm socket --tpm2 --tpmstate dir=/tmp/mytpm --server type=tcp,port=2321 \
             --ctrl type=tcp,port=2322 --flags not-need-init,startup-clear -d

# Connect tpm2-tools to swtpm
export TPM2TOOLS_TCTI="swtpm:port=2321"
tpm2_startup -c

# Verify properties, random numbers, and PCRs
tpm2_getcap properties-fixed | head -n 12
tpm2_getrandom --hex 16
tpm2_pcrread sha256:0,1,2,7,16
```

Observations:

1. tpm2_getcap properties-fixed confirmed the TPM family is 2.0 and revision is 1.83.
2. tpm2_getrandom produced 16 random hex bytes (78fc0e71d8f25537a72803396077f1d1), verifying that the random number generator is operational.
3. tpm2_pcrread showed that the SHA-256 PCR bank (including PCR 16) is clean and initialized to thirty-two zeros.



2. TPM Key Creation and Data Sealing

Commands Executed: Data Sealing refers to encryption of data in such a way that it can only be decrypted by the Trusted Platform Module (TPM) when the platform is in an expected state (PCR values). The process involved creating a secret payload, a primary key, and sealing the data to the clear state of PCR 16:

```
# 1. Create a secret data file
echo "SECURE_DATA_PRIYAJIT_23011102068" > secret_payload.txt
cat secret_payload.txt

# 2. Create the Storage Primary Key under the Owner hierarchy
tpm2_createprimary -C o -g sha256 -G rsa -c primary.ctx

# 3. Read current PCR 16 value and create a policy from it
tpm2_pcrread sha256:16 -o pcr16_val.bin
tpm2_createpolicy --policy-pcr -l sha256:16 -f pcr16_val.bin -L pcr16.policy
xxd -p pcr16.policy

# 4. Seal the secret file to the policy and load into TPM memory
tpm2_create -C primary.ctx -u sealed.pub -r sealed.priv -i secret_payload.txt -L
pcr16.policy
tpm2_flushcontext -t
tpm2_load -C primary.ctx -u sealed.pub -r sealed.priv -c sealed.ctx
```

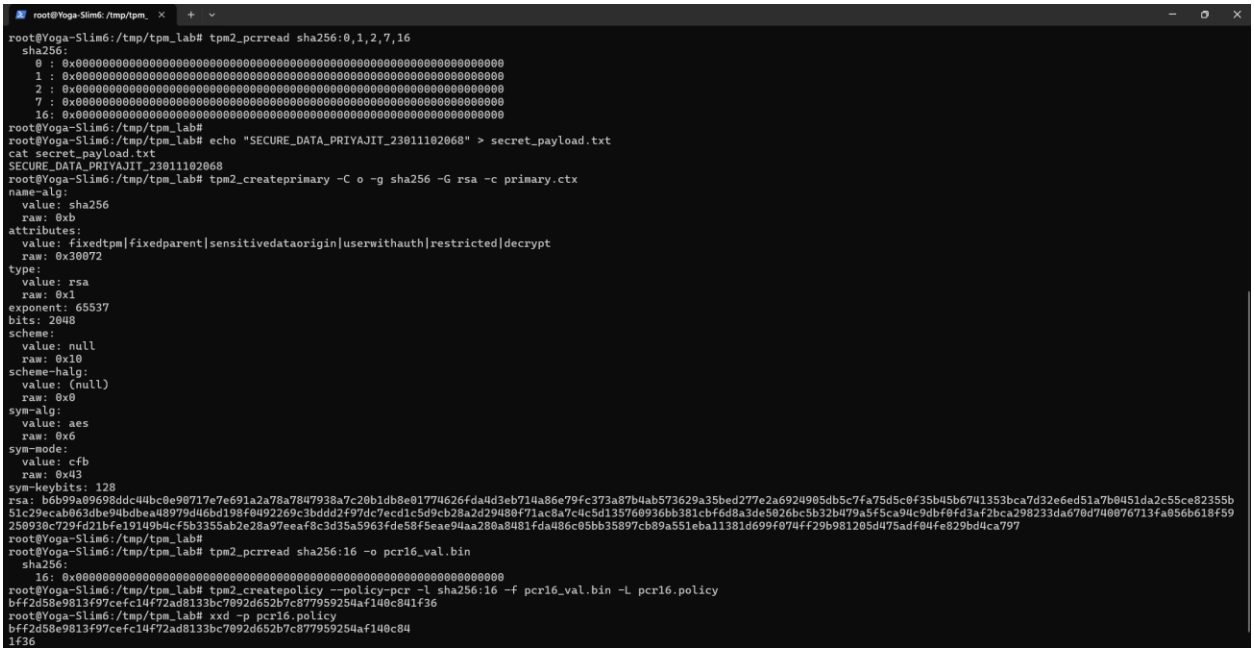
Observations:

1. The secret payload file was created with the student text.

2. An RSA 2048-bit primary key was created under handle primary.ctx.

3. The policy tool read PCR 16 (all zeros) and generated an authorization policy digest: bff2d58e9813f97cefc14f72ad8133bc7092d652b7c877959254af140c841f36.

4. tpm2_create generates the public and private sealed files. After flushing the temporary slots, tpm2_load loaded the sealed object into the TPM memory as sealed.ctx.



```
root@Yoga-Slim6:/tmp/tpm_lab# tpm2_pcrread sha256:0,1,2,7,16
  sha256:
    0 : 0x0000000000000000000000000000000000000000000000000000000000000000
    1 : 0x0000000000000000000000000000000000000000000000000000000000000000
    2 : 0x0000000000000000000000000000000000000000000000000000000000000000
    7 : 0x0000000000000000000000000000000000000000000000000000000000000000
    16: 0x0000000000000000000000000000000000000000000000000000000000000000
root@Yoga-Slim6:/tmp/tpm_lab#
root@Yoga-Slim6:/tmp/tpm_lab# echo "SECURE_DATA_PRIYAJIT_23011102068" > secret_payload.txt
cat secret_payload.txt
SECURE_DATA_PRIYAJIT_23011102068
root@Yoga-Slim6:/tmp/tpm_lab# tpm2_createprimary -C o -g sha256 -G rsa -c primary.ctx
name-alg:
  value: sha256
  raw: 0xb
attributes:
  value: fixedtpm|fixedparent|sensitivedataorigin|userwithauth|restricted|decrypt
  raw: 0x30072
type:
  value: rsa
  raw: 0x1
exponent: 65537
bits: 2048
scheme:
  value: null
  raw: 0x10
scheme-halg:
  value: (null)
  raw: 0x0
sym-alg:
  value: aes
  raw: 0x6
sym-mode:
  value: cfb
  raw: 0x43
sym-keybits: 128
rsa: b6b99a09698ddc44bc0e90717e7e691a2a78a7847938a7c20b1db8e01774626fda4d3eb714a86e79fc373a87b4ab573629a35bed277e2a6924905db5c7fa75d5c0f35b45b6741353bca7d32e6ed51a7b0451da2c55ce82355b
51c29ecab063dbe94bdbea48979d46bd198f0492269c3bddd2f97dc7ecd1c5d9cb28a2d29480f71ac8a7c4c5d135760936bb381cbf6d8a3de5026bc5b32b479a5f5ca94c9dbf0fd3af2bca298233da670d740076713fa056b618f59
250930c729fd21bfe19149b4cf5b3355ab2e28a97eeaf8c3d35a5963fde58f5eae94aa280a8481fda486c05bb35897cb89a551eba11381d699f074ff29b981205d475adf04fe829bd4ca797
root@Yoga-Slim6:/tmp/tpm_lab#
root@Yoga-Slim6:/tmp/tpm_lab# tpm2_pcrread sha256:16 -o pcr16_val.bin
  sha256:
    16: 0x0000000000000000000000000000000000000000000000000000000000000000
root@Yoga-Slim6:/tmp/tpm_lab# tpm2_createpolicy --policy-pcr -l sha256:16 -f pcr16_val.bin -L pcr16.policy
bff2d58e9813f97cefc14f72ad8133bc7092d652b7c877959254af140c841f36
root@Yoga-Slim6:/tmp/tpm_lab# xxd -p pcr16.policy
bff2d58e9813f97cefc14f72ad8133bc7092d652b7c877959254af140c84
1f36
```

```
root@Yoga-Slim6:/tmp/tpm_lab# tpm2_create -C primary.ctx -u sealed.pub -r sealed.priv -i secret_payload.txt -L pcr16.policy
name-alg:
  value: sha256
  raw: 0xb
attributes:
  value: fixedtpm|fixedparent
  raw: 0x12
type:
  value: keyedhash
  raw: 0x8
algorithm:
  value: null
  raw: 0x10
keyedhash: 4f84d24d45c67085b873a003dd15e9415d9e6e45b1ce947842e40c8abd0b79ab
authorization policy: bff2d58e9813f97cefc14f72ad8133bc7092d652b7c877959254af140c841f36
root@Yoga-Slim6:/tmp/tpm_lab# tpm2_load -C primary.ctx -u sealed.pub -r sealed.priv -c sealed.ctx
WARNING:esys:src/tss2-esys/api/Esys_Load.c:324:Esys_Load_Finish() Received TPM Error
ERROR:esys:src/tss2-esys/api/Esys_Load.c:112:Esys_Load() Esys Finish ErrorCode (0x00000902)
ERROR: Esys_Load(0x902) - tpm:warn(2.0): out of memory for object contexts
ERROR: Unable to run tpm2_load
root@Yoga-Slim6:/tmp/tpm_lab# tpm2_flushcontext -t
tpm2_load -C primary.ctx -u sealed.pub -r sealed.priv -c sealed.ctx
name: 000b865c286cb821c8b7892c383833ee61f4fa6db29954ae73067eef34640b3bf73e
root@Yoga-Slim6:/tmp/tpm_lab#
```

3. Data Unsealing and Policy Verification

Two tests for verifying policy enforcement were performed. The first test was to unseal when the PCR matches the policy. The second one was to try to unseal after tampering with PCR.

Phase 3A: Successful Unsealing under Matching Policy:

As PCR 16 was not altered and is all zeros, I would get a matching session digest from the policy digest that was sealed in the sealed object. TPM would be able to decrypt and unseal the data:

```
# Start policy session and evaluate PCR 16
tpm2_startauthsession -S session.ctx –policy-session
tpm2_policypcr -S session.ctx -l sha256:16

# Unseal the secret
tpm2_unseal -p session:session.ctx -c sealed.ctx -o unsealed_output.txt
tpm2_flushcontext session.ctx
cat unsealed_output.txt
```

Observation: The result was that the command tpm2_policypcr resulted in a digest of bff2d58e… which matched the sealed policy. The data was unsealed to unsealed_output.txt and displayed the original text SECURE_DATA_PRIYAJIT_23011102068.

```
root@Yoga-Slim6:/tmp/tpm_lab#
root@Yoga-Slim6:/tmp/tpm_lab# tpm2_startauthsession -S session.ctx --policy-session
root@Yoga-Slim6:/tmp/tpm_lab# tpm2_policypcr -S session.ctx -l sha256:16
bff2d58e9813f97cefc14f72ad8133bc7092d652b7c877959254af140c841f36
root@Yoga-Slim6:/tmp/tpm_lab# tpm2_unseal -p session:session.ctx -c sealed.ctx -o unsealed_output.txt
root@Yoga-Slim6:/tmp/tpm_lab#
root@Yoga-Slim6:/tmp/tpm_lab# tpm2_flushcontext session.ctx
root@Yoga-Slim6:/tmp/tpm_lab# cat unsealed_output.txt
SECURE_DATA_PRIYAJIT_23011102068
root@Yoga-Slim6:/tmp/tpm_lab#
root@Yoga-Slim6:/tmp/tpm_lab# cat unsealed_output.txt
SECURE_DATA_PRIYAJIT_23011102068
```

Phase 3B: Demonstration of Unsealing Rejection (Tampered State): To simulate either an attack or tampered bootloader, PCR 16 was extended with an arbitrary value tpm2_pcrextend. An unsealing trial was made:

```
# Tamper PCR 16 by extending it
tpm2_pcrextend
16:sha256=f1d2d2f924e986ac86fdf7b36c94bcdf32beec158525b6a3832c3bb47565ef02
tpm2_pcrread sha256:16

# Reset slots and reload sealed object
tpm2_flushcontext -t
tpm2_flushcontext -l
tpm2_load -C primary.ctx -u sealed.pub -r sealed.priv -c sealed.ctx

# Attempt unseal under tampered state
tpm2_startauthsession -S session.ctx –policy-session
tpm2_policypcr -S session.ctx -l sha256:16
tpm2_unseal -p session:session.ctx -c sealed.ctx -o unsealed_tampered.txt
tpm2_flushcontext session.ctx
```

Observations and Error Output:

Extending PCR 16 changed its value from all zeros to 0x897BDEB41C92E1BD8EFBC4645F6ECEB0CC2A2B175CB437ACD5D5B48CB8300AF9.

The policy session evaluates to a new digest 5f8f2df4… which differs from the expected policy bff2d58e…, thus TPM denied the request and caused an error:

```
ERROR: Esys_Unseal(0x99D) – tpm:session(1):a policy check failed
ERROR: Unable to run tpm2_unseal
```

This indicates that the TPM correctly recognized the tampered state of the platform.

```
root@Yoga-Slim6:/tmp/tpm_lab# cat unsealed_output.txt
SECURE_DATA_PRIYAJIT_23011102068
root@Yoga-Slim6:/tmp/tpm_lab# tpm2_pcrextend 16:sha256=f1d2d2f924e986ac86fdf7b36c94bcdf32beec158525b6a3832c3bb47565ef02
root@Yoga-Slim6:/tmp/tpm_lab# tpm2_pcrread sha256:16
  sha256:
    16: 0x897BDEB41C92E1BD8EFBC4645F6ECEB0CC2A2B175CB437ACD5D5B48CB8300AF9
root@Yoga-Slim6:/tmp/tpm_lab# tpm2_startauthsession -S session.ctx --policy-session
root@Yoga-Slim6:/tmp/tpm_lab# tpm2_policypcr -S session.ctx -l sha256:16
5f8f2df4f3d6ce56d07767253421976686897669fe449b5a1562b2ef5b27ed30
root@Yoga-Slim6:/tmp/tpm_lab# tpm2_unseal -p session:session.ctx -c sealed.ctx -o unsealed_tampered.txt
WARNING:esys:src/tss2-esys/api/Esys_ContextLoad.c:279:Esys_ContextLoad_Finish() Received TPM Error
ERROR:esys:src/tss2-esys/api/Esys_ContextLoad.c:93:Esys_ContextLoad() Esys Finish ErrorCode (0x00000902)
ERROR: Esys_ContextLoad(0x902) - tpm:warn(2.0): out of memory for object contexts
ERROR: Incorrect handle value, got: "sealed.ctx", expected a handle number
ERROR: Unable to read PEM from provided BIO/file
ERROR: Unable to fetch public/private portions of TSS PRIVKEY
ERROR: Cannot make sense of object context "sealed.ctx"
ERROR: Invalid item handle authorization
ERROR: Unable to run tpm2_unseal
root@Yoga-Slim6:/tmp/tpm_lab# tpm2_flushcontext session.ctx
root@Yoga-Slim6:/tmp/tpm_lab# tpm2_flushcontext -t
tpm2_flushcontext -l
root@Yoga-Slim6:/tmp/tpm_lab# tpm2_load -C primary.ctx -u sealed.pub -r sealed.priv -c sealed.ctx
name: 000b865c286cb821c8b7892c383833ee61f4fa6db29954ae73067eef34640b3bf73e
root@Yoga-Slim6:/tmp/tpm_lab# tpm2_startauthsession -S session.ctx --policy-session
tpm2_policypcr -S session.ctx -l sha256:16
5f8f2df4f3d6ce56d07767253421976686897669fe449b5a1562b2ef5b27ed30
root@Yoga-Slim6:/tmp/tpm_lab# tpm2_unseal -p session:session.ctx -c sealed.ctx -o unsealed_tampered.txt
WARNING:esys:src/tss2-esys/api/Esys_Unseal.c:295:Esys_Unseal_Finish() Received TPM Error
ERROR:esys:src/tss2-esys/api/Esys_Unseal.c:98:Esys_Unseal() Esys Finish ErrorCode (0x0000099d)
ERROR: Esys_Unseal(0x99D) - tpm:session(1):a policy check failed
ERROR: Unable to run tpm2_unseal
root@Yoga-Slim6:/tmp/tpm_lab# tpm2_flushcontext session.ctx
root@Yoga-Slim6:/tmp/tpm_lab#
```

4. Analysis and Documentation

Explanation of Commands and Error Code 0x99D: In Task 3A, after successful unsealing of the PCR bank, it was discovered that the session digest calculated from PCR 16 was equal to bff2d58e..., which matched the authorization policy digest built into the sealed object. However, when Task 3B was executed, and PCR 16 was extended, the session digest changed to 5f8f2df4...

Since the session digest does not match the sealed value, the TPM returned error code 0x99D (TPM2_RC_POLICY_FAIL). This error code is specific to the TPM 2.0 specification and states that a policy check has failed. In this case, the TPM hardware refused to decrypt the private file. As a result, the secret data remained protected.

How PCR Sealing Protects Data in Practice: PCR-bound data sealing provides much better protection of sensitive computer data than standard passwords:

1. Laptop Theft and Offline Attacks: If a laptop is stolen but BitLocker or LUKS disk encryption is PCR-sealed, it will not be possible to simply boot the machine into another OS from a USB drive to read the files. This is because booting into another OS will change PCR 0, 4, and 7, which will cause the TPM to refuse to release the disk's decryption key.

2. Protection Against Boot Malware: Malware or rootkits which modify the BIOS, bootloader, or kernel will cause PCR measurement mismatches, meaning that the TPM will refuse to release the decryption key.

3. Unattended Cloud Decryption: A server in a remote data center can be booted up and automatically decrypted without a password by an administrator, as the TPM will only release the decryption key if the boot process has not been tampered with.

5. Conclusion

I confirm that all the requirements of the CIA-2 standard were met. First, using the swtpm and tpm2-tools software, I created a TPM 2.0 object, then I sealed it to a PCR policy. Then, by meeting the PCR policy, I was able to unseal the object, and the secret text was displayed. Finally, by extending PCR 16 to simulate a tamper, the TPM refused to unseal the object with error code 0x99D. This whole process shows how hardware-enabled security is used to protect valuable data from unauthorized disclosure.